

ForgetEval: Benchmarking the Forgetting Axis of Agent Memory Systems

Dongxu Yang

DeepLethe

2026-05-13

Abstract

Agent-memory benchmarks measure retrieval recall: *how often is the right fact in the top-k?* Production deployments fail in the inverse direction. A password the user rotated three months ago is still recommended; a GDPR-deleted customer is still in the candidate pool; a user’s job title persists in three contradictory versions across sessions. These are not recall failures — the system retrieved the fact correctly — they are **forgetting** failures: the inability to direct a memory store to no longer surface specific content. No open benchmark measures this axis.

We propose FORGETEVAL, a methodology and 1000-case English benchmark (plus 250-case Chinese and Japanese extensions) for the forgetting axis. FORGETEVAL decomposes forgetting into five structural families — **supersession**, **decay**, **amnesia**, **purge**, **drift** — and operationalizes each as template-generated cases scored by deterministic substring match over top-k recall. No LLM judge is required, so the protocol is fully reproducible. A six-method **Adapter** Protocol lets any memory system enter the evaluation in roughly 130 lines of glue code, with honest **N/A scoring** for missing primitives.

We accompany the benchmark with LETHE, a reference implementation organized around a single scalar axis $\text{depth} \in \mathbb{R}$. Every memory state — pinned, surface, sinking, submerged, erased — is a value on this axis; every forgetting operation is a force on it. We give a formal specification of the depth model with four propositions characterizing monotonicity, idempotence, irreversibility of purge, and event-log determinism. On FORGETEVAL at 1000 cases, LETHE scores 99.3% (993/1000); $\text{mean} \pm \sigma$ across five scale=250 seeds is $99.1 \pm 0.7\%$. MEM0 v2.0.2 scores 88.8%, collapsing on amnesia (70%) and purge (75%) — a 10-point overall gap, more than ten times LETHE’s per-seed σ . MEMPALACE returns 0/1000 because the required primitives are not exposed. A second deterministic baseline — LangChain’s LangGraph **InMemoryStore** — scores 99.5% on the same template suite, confirming that vector-store-backed systems with adaptive eviction near-saturate this axis. We also publish a **112-case hand-crafted adversarial layer** (FORGETEVAL-Adv, 10 attack categories) where all three deterministic systems converge to 62–68%; overall Wilson 95% intervals overlap, so the bench reads as a *trade-off*, not a winner. Per-category significance separates the systems: LETHE beats MEM0 on `prefix_collision` (100% vs. 50%, $p < 0.05$) thanks to its lexical-precise purge; MEM0 beats LETHE on `cross_lingual_identifier` (50% vs. 0%, $p < 0.05$) thanks to multilingual-embedding soft matching. LETHE is $\sim 11\times$ faster per case than MEM0 on the same suite ($\sim 48\text{ms}$ vs. $\sim 527\text{ms}$). For attack categories that need genuine semantic understanding — `compound_fact` (all 3 systems 0/8) and `identifier_obfuscation` under script variation — the **LetheAdapter** exposes an *LLM-optional hook* that routes those two specific decisions to a model at mutation time; the recall hot path stays LLM-free. On the conventional retrieval benchmark LongMemEval-S, LETHE also attains $R@5 = 97.4\%$ vs. MEMPALACE’s 96.6%, with the $R@1$ gap six times wider than the $R@5$ gap.

FORGETEVAL and LETHE are released under MIT at <https://github.com/deeplethe/lethe>. We invite cross-publication on this axis from other memory frameworks.

Keywords: LLM agents; long-term memory; benchmark; forgetting; machine unlearning; GDPR.

1 Introduction

Every memory framework for LLM agents is racing in the same direction. MEM0 (?) promises perfect recall through automatic update/delete/no-op routing. MEMPALACE (?) promises verbatim retention as a feature. MemGPT and its successor Letta (??) delegate memory paging to the LLM itself. Zep (?), HippoRAG (?), A-MEM (?), and Cognee (?) each propose graph-, episode-, or zettelkasten-style variants. They compete on a shared battery: R@k on LongMemEval (?), MTEB-Retrieval (?), or internal “LOCOMO”-style conversational suites. The unstated common assumption is that the failure mode worth engineering against is **losing a fact**.

In production, the failure mode is the opposite.

- A password the user rotated three months ago is still suggested to the LLM as a candidate credential.
- A customer who exercised the GDPR Article 17 right to erasure (?) remains in the recommendation candidate pool because the embedding was never deleted.
- A user’s job title — updated through six conversational turns over two years — surfaces in three contradictory versions depending on which session the agent is restored from.
- A one-time verification code from a session two weeks ago lives permanently next to the user’s stable preferences, drifting through recall results.

None of these are absence-of-recall failures. Recall worked; the fact was retrieved. They are the failure of *forgetting*: the system retrieved the right thing too well, when the application needed it to retrieve nothing.

The retrieval-only benchmarking culture has a measurable consequence. Among three open-source memory systems we evaluate, none score above 90 % across all five forgetting families, and one scores zero because the relevant primitives are not exposed by the public API at all. The gap is not a tail-of-the-distribution effect: it shows up at the very first case in two of five families for the systems we test.

This paper argues that **the forgetting axis is benchmarkable with the same rigor as recall**, and demonstrates that the axis is **satisfiable** with the right architecture. Concretely, our contributions are:

1. **ForgetEval (§??–§??)**: a methodology that decomposes forgetting into five structural families — supersession, decay, amnesia, purge, drift — operationalized as 1000 template-generated cases scored by deterministic substring match over top-k recall. Pass/fail is computable without an LLM judge. An **Adapter** Protocol lets any memory system enter the evaluation in ~130 lines of glue, with honest **N/A scoring** for missing primitives.
2. **Lethe (§??)**: a reference open-source memory store organized around a single scalar depth $\in \mathbb{R}$ with four forgetting operations defined as forces on depth. We give a precise specification of the model (§??), a worked example of its event-log semantics, and describe the cryptographic Ed25519 purge receipts (§??) that make GDPR-grade deletion auditable.
3. **Empirical comparison (§??)**: LETHE scores 99.3 % on FORGETEVAL at 1000 cases, identically at seed=42 and seed=43; $\sigma = 0.69\%$ over five seeds at scale=250. MEM0 v2.0.2 scores 88.8 % overall, with amnesia at 70 % and purge at 75 %. MEMPALACE scores 0 % because supersede / release / purge are not exposed. LETHE also surpasses MEMPALACE on LongMemEval-S, the conventional retrieval benchmark (R@5 97.4 % vs. 96.6 %; R@1 85.4 % vs. 80.6 % — the R@1 gap is six times wider than R@5).
4. **Failure analysis (§??)**: each FORGETEVAL score maps to a concrete production failure mode. A 70 % amnesia rate means three in ten right-to-be-forgotten requests leak fragments to siblings; a 75 % purge rate means one in four identifier-precise deletions either miss the target or take a neighbor with them.

The complete benchmark, adapter contract, and LETHE reference implementation are released under MIT at <https://github.com/deeplethe/lethe>. We invite teams maintaining other

memory frameworks to submit adapters; results will be cross-published on a public leaderboard.

The remainder of the paper is organized as follows. §?? surveys related work in memory benchmarks, agent memory architecture, and the psychology and machine-unlearning literatures on forgetting. §?? specifies FORGETEVAL’s five families and their templated structure. §?? documents the **Adapter** Protocol and the rationale for honest N/A scoring. §?? describes LETHE’s depth model with formal propositions. §?? reports experimental results: headline numbers, seed variance, distractor robustness, component ablations, latency, the conventional retrieval benchmark, multilingual, and a system-architecture summary. §?? translates each score into a production failure mode. §??–§?? cover limitations and threats to validity. §?? is the reproducibility statement. §?? discusses why this works, when it fails, and where to go next. §?? concludes.

2 Related Work

We position FORGETEVAL against four lines of work.

Memory benchmarks. LongMemEval (?) is the most prominent conversational-memory benchmark. It comprises 500 questions across seven categories — single-session-user, single-session-assistant, single-session-preference, multi-session, knowledge-update, temporal-reasoning, and abstention — at two scales: LongMemEval-S (~115 k tokens / 30–40 sessions per question) and LongMemEval-M (~1.5 M tokens / ~500 sessions per question). Every category is a recall question: “given a long history h , can the system surface the answer?” Even the *knowledge-update* category — which tests whether the system tracks an evolved fact such as a new job title — ultimately asks whether the *latest* fact is retrieved; it does not test whether the *superseded* fact has been removed. MTEB (?) and BEIR (?) are stateless retrieval benchmarks at the embedding-model layer, scoring ranking quality over a static corpus with no notion of write/update/delete operations. MEM0’s paper (?) introduces LOCOMO scoring (10 long multi-session dialogues evaluated by an LLM judge), and Zep’s paper (?) reports on LongMemEval and DMR (Deep Memory Retrieval); none of these benchmarks probe commanded forgetting.

Open-source memory frameworks. MEM0 (?) uses an LLM-driven router that classifies each new fact into ADD / UPDATE / DELETE / NoOp; DELETE is *contradiction-triggered overwrite* of an existing memory, not a user-commanded purge. On LOCOMO, MEM0 reports an overall LLM-judge score of 66.88 % (MEM0^g, the graph variant, 68.44 %), with 91 % p95 latency reduction and ~90 % token savings vs. full-context baselines. MemGPT (?) (now Letta (?)) introduced a tiered “OS-inspired” architecture: a fixed-size *main context*, plus unbounded *recall* and *archival* storage, with the LLM itself calling functions to page between them. The 2023 MemGPT paper does not define an `archival_memory_delete` tool; the current Letta platform exposes block-level deletion via its control-plane API, but no in-agent forget primitive and no audit trail of replacements. Zep (?) is built on Graphiti, a temporally-aware knowledge graph: each edge carries four timestamps (t'_{created} , t'_{expired} , t_{valid} , t_{invalid}), and contradictory edges are *invalidated* (t_{invalid} set) rather than deleted, preserving auditable history. Zep reports up to +18.5 % over baseline on LongMemEval with ~90 % latency reduction. Edge invalidation in Zep is contradiction-driven inside the system, not user-issued — it is the closest landscape precedent to forget-on-command, but it does not meet that bar. HippoRAG (?) models long-term memory with personalized PageRank over an OpenIE-extracted knowledge graph, reporting up to +20 % over SOTA on multi-hop QA. HippoRAG is append-only: there is no documented update, delete, or supersession primitive. A-MEM (?) applies the Zettelkasten method, generating LLM links between notes and updating *neighbor link-context* when new notes arrive — an UPDATE on link metadata, not on fact content; no deletion or supersession is exposed. Cognee (?) is an open-source graph-RAG library whose documentation indicates that graph updates are

implemented as destructive chunk-replace (delete and re-insert); a “generalized delete logic” is documented as in progress. No peer-reviewed evaluation is published. MEMPALACE (?) takes the opposite position from MEM0: verbatim retention is the feature, not a defect. Its API exposes ingest and search with no delete or update operations whatsoever — our N/A scoring on three families correctly reflects this design choice rather than penalizing implementation quality.

Forgetting in human episodic memory. Ebbinghaus’s forgetting curve (?) is the foundational quantitative model. Bjork’s directed-forgetting paradigm (?) showed that humans can be instructed to forget specific items, the empirical analog of our **decay** and **release** families; Anderson and Spellman (?) documented retrieval-induced forgetting, where retrieving fact *A* suppresses related fact *B* — an analog of our **amnesia** width-control problem. FORGETEVAL is not a psychology benchmark, but the family decomposition is partially motivated by which kinds of forgetting humans can be reliably *instructed* to perform.

Machine unlearning. Cao and Yang (?) introduced the term and proposed a summation-form training pipeline making point removal asymptotically faster than retraining. Bourtole et al. (?) introduced SISA (Sharded, Isolated, Sliced, Aggregated) — partition the training corpus into shards and slices, train one constituent model per shard, and on a deletion request retrain only the affected shard from the nearest pre-slice checkpoint. This literature targets the *model-weights* layer: its goal is statistical indistinguishability from a model trained without the forgotten datum (driven by GDPR Article 17, copyright, privacy). FORGETEVAL is positioned adjacent: we measure forgetting in the *agent-memory* layer — a retrieval-augmented store sitting beside the model — where the forgotten unit is an inference-time memory item, not a training datum. The two literatures share vocabulary but address different system layers; a production system that handles GDPR end-to-end needs both.

A condensed comparison of memory systems on the dimensions that matter for forgetting is given in Table ?? (§??).

3 The ForgetEval Methodology

FORGETEVAL is built on the observation that *forgetting* is not one operation but a family of operations with distinct production semantics. We isolate five.

3.1 The five families

Each family probes one structural property a memory system must exhibit if it is to be safe under specific production conditions.

Supersession. When a fact changes — job, address, preference — the new fact must win top-k recall and the old fact must not appear. The default failure mode is to leave both facts live with similar recall scores; the agent then confabulates between the two.

Example. Setup: “User works at Stripe.” Mutation: `supersede("user works at", "User now works at Anthropic")`. Query: “Where does the user work?” Pass: top-k contains “Anthropic” and does not contain “Stripe”.

Decay. A released fact — TTL expired, OTP consumed, intent cancelled — must stay out of top-k. Decay is the simplest forgetting operation but exposes whether the system has any primitive for “this is no longer relevant.”

Example. Setup: “Session OTP: 887766.” Mutation: `release("OTP for current session")`. Query: “What is the OTP?” Pass: top-k does not contain “887766”.

Amnesia. Forget every fact about one entity, while sibling entities survive. The hardest part is *width control*: too narrow forgets one fact out of five; too wide forgets unrelated siblings.

Example. Setup: “Dana works at Stripe. Dana lives in Berlin. Dana speaks Portuguese. Eve works at Notion.” Mutation: `release("everything about Dana")`. Query: “Tell me about the people.” Pass: top-k contains "Eve" but no "Dana" substring.

Purge. Hard-delete by identifier (email, API key, patient name). Unlike release, purge is non-reversible and *precise*: deleting `alice@acme.io` must not also delete `bob@acme.io` even though the two rows are semantically near-identical to an embedder. This is the operation GDPR Article 17, HIPAA, and SOC2 actually care about, and the one where semantic similarity is the wrong primitive.

Example. Setup: “Customer `alice@acme.io` bought 3 items. Customer `bob@acme.io` ordered yesterday.” Mutation: `purge("alice@acme.io")`. Query: “Show me customer `alice@acme.io`’s data.” Pass: top-k does not contain "`alice@acme.io`"; "`bob@acme.io`" is still retrievable from its own query.

Drift. A chain of supersedes over time — the topic gets updated several times, only the most recent belief wins, every intermediate must be unreachable. Drift is the long-horizon failure mode: a system might handle one supersede correctly but accumulate stale intermediates over N .

Example. Setup and mutations: “User started at Google in 2020” → supersede → “moved to Meta in 2022” → supersede → “at Anthropic in 2025.” Query: “Where does the user work?” Pass: top-k contains "Anthropic" and contains neither "Google" nor "Meta".

The five families are not orthogonal — drift is supersession $\times N$, and amnesia is several releases bundled — but they probe distinct architectural choices. Supersession asks whether the system has a *notion of replacement*; decay asks whether it has a *notion of relevance-going-to-zero*; amnesia asks whether it can *scope width*; purge asks whether it has an *identifier-precise hard delete*; drift asks whether the system is *closed under repeated supersession*.

3.2 Case anatomy

Every test case is a single dataclass:

```
@dataclass
class GeneratedCase:
    id: str # e.g. "supersession__job__042"
    family: str # one of the five families
    setup_facts: list[str] # inscribed in order, mixed w/
        distractors
    mutations: list[tuple] # ("supersede", old_q, new_text)
        # ("release", query)
        # ("purge", query)
    final_query: str # the question posed at recall time
    must_contain: list[str] # substrings that MUST appear in top-k
    must_not_contain: list[str] # substrings that MUST NOT appear
```

The run loop is six lines. No LLM judge, no fuzzy matching.

```
adapter.reset()
for fact in case.setup_facts: adapter.inscribe(fact)
for mut in case.mutations: getattr(adapter, mut[0])(*mut[1:])
top = adapter.recall_texts(case.final_query, k=10)
blob = " ".join(top).lower()
passed = (all(s.lower() in blob for s in case.must_contain) and
          all(s.lower() not in blob for s in case.must_not_contain))
```

A case passes if both sides hold: the right thing was kept *and* the wrong thing was removed.

Table 1: Sub-templates per family. The hard sub-templates (`long_form`, `one_of_many`, `many_facts`, `many_similar`, `long_chain`) provide the most discriminative signal.

Family	Sub-templates
supersession	job, theme, diet, long_form
decay	otp, verification_code, flight_cancelled, one_of_many
amnesia	person, multi, topic, many_facts
purge	api_key, phi, gdpr, many_similar
drift	jobs, address, color, long_chain

Table 2: Standard generation profiles.

Profile	n / family	distractors	total cases	wall (Lethe, MiniLM)
smoke	10	4	50	~ 4 s
standard	50	4	250	~ 20 s
full	200	4	1000	~ 80 s
stress	50	50	250	varies

3.3 Distractors

Each case mixes the target facts with N filler facts unrelated to the target’s tokens (office-life trivia: meeting rooms, coffee machines, fire drills). Two reasons:

1. **Forces real retrieval.** Without distractors, top-k is trivially correct: only the target was inscribed.
2. **Stress-tests width control.** A naive `release(query)` that over-evicts catches the wrong neighbors first; distractors widen the candidate pool so the bug shows up.

The default $N = 4$ matches the density of a short conversational context. We sweep $N \in \{4, 10, 25, 50, 100\}$ in §?? to stress adaptive-threshold heuristics.

3.4 Sub-templates and entity pools

Each family uses four sub-templates the generator cycles through, producing case-level variety (Table ??).

Sub-templates are pure functions of an RNG seed and short entity pools of real proper nouns: NAMES, COMPANIES, CITIES, COLORS, DIETS, THEMES, LANGUAGES, BOOKS, HOBBIES, STREETS, EMAIL_DOMAINS, CONDITIONS.

3.5 Generation protocol

- **Deterministic.** `generate(n_per_family, seed=42, distractors=4)` returns the same $5n$ cases byte-for-byte on every call with the same seed. No floating-point sources, no time-of-day, no LLM calls in the generator.
- **No training-set contamination.** Entity pools are short, common proper nouns (Alice, Stripe, Berlin); no public benchmark text is reused.
- **Linear scaling.** Cases per family scale linearly with `n_per_family`; the default profiles are summarized in Table ??.

3.6 Scoring

Per case: pass (1) or fail (0). No partial credit.

Per family: $\text{rate}_f = \text{pass}_f / |F|$.

Overall: $\text{rate} = \sum_f \text{pass}_f / \sum_f |F|$, plus the number of cases marked **N/A** (capability not implemented; counted as fail in the overall rate, surfaced separately in the report).

The runner distinguishes three outcomes:

- ✓ pass — $\text{must_contain} \subseteq \text{top-k blob}$ and $\text{must_not_contain} \cap \text{top-k blob} = \emptyset$
- × fail — the adapter responded, but the conditions weren’t met
- · N/A — the adapter raised `NotImplementedError`; the operation isn’t supported

N/A is honest, not generous. A memory system that cannot supersede is strictly worse than one that supersedes badly, because the latter at least exposes the primitive that the caller can compose with.

4 The Adapter Contract

To evaluate a new memory system, the maintainer writes an adapter implementing a six-method typing.`Protocol`:

```
class Adapter(Protocol):
    name: str

    # Mandatory.
    def reset(self) -> None: ...
    def inscribe(self, text: str) -> int | str: ...
    def recall_texts(self, query: str, k: int = 5) -> list[str]: ...

    # Optional - each NotImplementedError maps to N/A on the relevant
    # families, which is the honest reflection of the system's API.
    def supersede(self, old_query: str, new_text: str) -> None: ...
    def release(self, query: str) -> int: ...
    def purge(self, query: str) -> int: ...
```

Three methods are mandatory (`reset`, `inscribe`, `recall_texts`); three are optional, and each `NotImplementedError` correctly maps to N/A on the relevant families. The three adapters shipped in this paper (LETHE, MEM0, MEMPALACE) are each under 130 lines.

A design property we emphasize: the contract treats *missing primitives* as a first-class outcome. This matters in two ways. First, it lets FORGETEVAL distinguish *implemented and failed* from *not provided at all* — distinguishing 70% amnesia (MEM0) from 0% amnesia (MEMPALACE) reveals architectural choices rather than implementation quality. Second, it gives N/A scores a precise interpretation: a 0/200 means the API is missing, not that the benchmark was unfair.

5 Reference Implementation: Lethe

LETHE is a small, single-file embedded memory store (~700 lines of Python over SQLite + `sqlite-vec` (?) + FTS5) organized around a single scalar axis. We present it as a reference implementation, not as the central contribution: the benchmark is the contribution, and LETHE exists to show that the benchmark is satiable.

5.1 The depth model

We formalize LETHE’s state space. Let M be the set of memory items ever inscribed into a store, and let E be the totally-ordered set of events written to the append-only log. Each memory $m \in M$ has a text payload $\text{text}(m)$, an embedding $\mathbf{v}(m) \in \mathbb{R}^d$, and a depth function $\delta_m : E \rightarrow \mathbb{R} \cup \{+\infty, \text{void}\}$ giving the depth after each event applicable to m . We write $\delta_m^{(t)}$ for the depth of m after event t .

Table 3: Operations as forces on depth.

Operation	Transition for target m
<code>inscribe(text)</code>	$\delta \mapsto 1.0$ (initial value)
<code>consolidate()</code>	$\delta \mapsto \delta \cdot e^{-\Delta t/S(m)}$ if $\delta \in (0, +\infty)$, identity otherwise
<code>surrender(decay, m)</code>	$\delta \mapsto \delta/2$ if $\delta \neq +\infty$, identity otherwise
<code>surrender(release, m)</code>	$\delta \mapsto 0$
<code>surrender(supersede, m, text')</code>	$\delta_m \mapsto 0$; insert new m' with $\delta_{m'} = 1.0$; record edge (m, m')
<code>pin(m)</code>	$\delta \mapsto +\infty$
<code>unpin(m)</code>	$\delta \mapsto 1.0$
<code>surrender(purge, m)</code>	$\delta \mapsto \text{void}$ (row deleted from <code>memory</code> , <code>memory_vec</code> , <code>memory_fts</code>)

Definition 1 (Memory state). *At event time t , memory m is in exactly one of the following states:*

- **pinned:** $\delta_m^{(t)} = +\infty$
- **surface / sinking:** $\delta_m^{(t)} \in (0, +\infty)$
- **submerged:** $\delta_m^{(t)} = 0$
- **erased:** $\delta_m^{(t)} = \text{void}$ (row no longer exists)

Erased is the only terminal state.

Definition 2 (Recall set). *The recall candidate set at time t is $R^{(t)} = \{m \in M : \delta_m^{(t)} > 0\}$ — that is, the pinned and surface/sinking memories. Submerged and erased memories are never returned by recall.*

Definition 3 (Operations as forces on depth). *Each public operation induces a depth transition shown in Table ??.*

The consolidation factor is $S(m) = \tau \cdot (1 + \log(1 + a_m))$ where τ is a time constant (default $\tau = 7$ days) and a_m is the access count of m — frequently-accessed memories decay slower.

Proposition 1 (Monotone decay of unpinned, unaccessed memories). *Under repeated `consolidate()` with no intervening `inscribe`, `surrender`, `pin`, or `recall` events, an unpinned memory’s depth satisfies $\delta_m^{(t)} \rightarrow 0$ as $t \rightarrow \infty$. More precisely, if a_m remains constant, $\delta_m^{(t)} = \delta_m^{(t_0)} \cdot e^{-(t-t_0)/S(m)}$.*

Proof. $S(m)$ is finite and positive; the exponential is strictly between 0 and 1; iterated application of a factor in $(0, 1)$ to a positive number is monotone and limits to 0. $\square$

Proposition 2 (Pin idempotence). *`pin` $\circ$ `consolidate` = `pin` and `pin` $\circ$ `pin` = `pin`. No force in $\{\text{decay}, \text{release}\}$ moves $+\infty$ off $+\infty$; only `surrender(purge)` and `unpin` can.*

Proof. By construction, `_apply_force` and `_set_depth` (except for `purge` and the `unpin` $\rightarrow 1.0$ transition) leave $+\infty$ fixed. $\square$

Proposition 3 (One-way purge). *Once $\delta_m = \text{void}$, no operation in the API moves δ_m off `void`. Purge is the only terminal state and is irreversible at the system layer.*

Proof. The row is deleted from all three tables (`memory`, `memory_vec`, `memory_fts`); subsequent operations target by rowid, and a deleted rowid matches nothing. The event log retains the purge event for audit, but the event log alone is not a basis for restoring the row (no operation inverts purge). $\square$

Proposition 4 (Event-log determinism). *For any pair of times $t_1 < t_2$, the depth $\delta_m^{(t_1)}$ is uniquely determined by the prefix of events with timestamp $\leq t_1$, irrespective of any later events. This is the precondition that makes the `recall(query, at=T)` time-travel API well-defined.*

Proof. Every state transition is logged with $(\text{depth}_{\text{before}}, \text{depth}_{\text{after}})$. The state of m at t_1 is exactly the $\text{depth}_{\text{after}}$ field of the latest event with $\text{memory_id} = m$ and $\text{timestamp} \leq t_1$. Later events do not appear in this query. $\square$

The schema implementing this is minimal:

```
CREATE TABLE memory (
  rowid          INTEGER PRIMARY KEY,
  text           TEXT,
  depth          REAL DEFAULT 1.0,
  created_at     REAL,
  last_access    REAL,
  access_count   INTEGER DEFAULT 0,
  meta           TEXT
);
CREATE INDEX idx_memory_depth ON memory(depth);
```

There are no `is_active` flags, `superseded_at` columns, or soft-delete booleans. All recall queries filter `WHERE depth > 0`.

5.2 The four forces on depth

Forgetting operations are forces that move depth (already introduced in Table ??). The unified axis means *every* state transition records the same shape of event:

```
CREATE TABLE event (
  id              INTEGER PRIMARY KEY AUTOINCREMENT,
  memory_id       INTEGER NOT NULL,
  kind            TEXT NOT NULL,
  depth_before    REAL,
  depth_after     REAL,
  timestamp       REAL NOT NULL,
  meta            TEXT
);
```

This is the property that makes time-travel and audit trivial: the state of any memory at time T is the most recent event with $\text{timestamp} \leq T$.

5.3 Two retrieval primitives

`recall()` exposes two retrieval modes:

- **Hybrid** (default). Reciprocal-Rank Fusion (?) of `sqlite-vec` cosine-similarity ranking and FTS5 BM25 ranking, with equal weights and Cormack–Clarke–Buettcher constant $K = 60$. Used for natural-language queries.
- **Lexical** (`lexical=True`). Pure BM25 over the FTS5 index. Used for identifier lookup — emails, API keys, patient names — where vector similarity blurs near-paraphrases that are semantically distinct. In our adapter, `purge` uses lexical recall exclusively for exactly this reason (§??).

5.4 Verifiable forgetting: Ed25519-signed purge receipts

For compliance use cases (GDPR, HIPAA, SOC2), a memory system being asked to delete data must produce an audit-grade proof of deletion. LETHE’s append-only event log makes this trivial.

`purge_with_receipt()` issues a `PurgeReceipt` containing: the purged row ids, SHA-256 hashes of the purged texts before deletion, the event-log IDs of the corresponding `purge` events, and the Merkle root of the full event log at the time of issuance. The receipt is signed with the LETHE instance’s Ed25519 (?) private key. A third party can later verify (a) which rows were

purged, (b) that the event log has not been tampered with since, and (c) that the receipt was issued by the legitimate operator — all without needing access to the deleted data itself.

Receipts are 32-byte-key signatures over a canonical CBOR payload; issuing or verifying one is $O(\log n)$ in the event-log size and takes sub-millisecond time on a modern machine.

5.5 Time-travel via append-only event log

Because every state transition is logged with `depth_before` and `depth_after`, the recall operation accepts an optional `at=T` parameter that returns the view that existed at T :

```
WITH state_at_t AS (
  SELECT e.memory_id, e.depth_after AS depth
  FROM event e
  WHERE e.timestamp <= ?
  AND NOT EXISTS (
    SELECT 1 FROM event e2
    WHERE e2.memory_id = e.memory_id
    AND e2.timestamp > e.timestamp
    AND e2.timestamp <= ?
  )
)
SELECT m.rowid, m.text, s.depth, ...
FROM memory_vec v
JOIN memory m ON m.rowid = v.rowid
JOIN state_at_t s ON s.memory_id = m.rowid
WHERE v.embedding MATCH ? AND s.depth > 0
ORDER BY v.distance;
```

This costs one extra join, has no extra storage requirement beyond the event log, and gives users a `lethe.recall(query, at=T)` API suitable for debugging “what did the agent see when it made decision X on Tuesday?”

6 Experimental Results

6.1 Setup

All experiments use LETHE’s default embedder (`sentence-transformers/all-MiniLM-L6-v2`, $d = 384$, ONNX runtime via `fastembed` (?) and run on a single laptop CPU (no GPU). MEM0 v2.0.2 uses the same embedder via its HuggingFace provider; MEMPALACE uses its default embedding stack. All systems are reset to an empty state before each test case. No LLM is invoked anywhere on the evaluation hot path.

We report:

- **ForgetEval** at scale=200 (1000 cases) and scale=50 (250 cases for faster sweeps), default distractors=4 unless stated.
- **Variance**: five seeds {42, 43, 44, 45, 46} at scale=50.
- **Distractor robustness**: distractors $\in \{4, 10, 25, 50, 100\}$ at scale=50, three seeds each.
- **LongMemEval-S**: 500 questions, MiniLM embedder, no LLM judge.
- **Multilingual**: scale=50 (250 cases) for English / Chinese / Japanese.

6.2 ForgetEval at 1000 cases (English, seed=42, distractors=4)

Table ?? is the headline result. The LETHE row is from a fresh re-run for this paper (`data/full_1000.json`); a second 1000-case run at seed=43 produced 993/1000 = 99.30% exactly (different within-family breakdown: amnesia 193/200, drift 200/200), confirming the headline across seeds at full scale. The MEM0 row is the value shipped with `bench/forgeteval`; we did not re-run MEM0 for this

Table 4: Headline FORGETEVAL scores at 1000 cases. 95 % Wilson intervals in brackets.

System	super	decay	amnesia	purge	drift	Overall
Lethe v1	200/200	200/200	195/200	200/200	198/200	993/1000 = 99.30 % [98.41, 99.69]
MEM0 v2.0.2	200/200	200/200	140/200	150/200	198/200	888/1000 = 88.80 % [86.65, 90.62]
MEMPALACE	0/200	0/200	0/200	0/200	0/200	0/1000 = 0.00 % (no primitives)

Table 5: Per-seed FORGETEVAL scores at scale=50 (250 cases per seed). Variance across seeds is $\sigma = 0.69\%$ on the overall metric; saturated families have $\sigma = 0$.

Seed	super	decay	amnesia	purge	drift	Overall
42	100	100	90	100	100	98.0 (245/250)
43	100	100	96	100	100	99.2 (248/250)
44	100	100	100	100	100	100.0 (250/250)
45	100	100	96	100	98	98.8 (247/250)
46	100	100	98	100	100	99.6 (249/250)
mean $\pm \sigma$	100 $\pm$ 0	100 $\pm$ 0	96.0 $\pm$ 3.4	100 $\pm$ 0	99.6 $\pm$ 0.8	99.12 $\pm$ 0.69

paper because the Qdrant on-disk index occasionally leaves stale embeddings across rapid resets, and a careful multi-seed MEM0 sweep is left to the public leaderboard.

MEMPALACE returns zero on every family because supersede/release/purge all raise `NotImplementedError` in its adapter — its API exposes only ingest and search. MEM0 maintains 100 % on supersession, decay, and drift (each composable from its `delete` and `add`) but drops to 70 % on amnesia and 75 % on purge, the two families that probe *width control* and *identifier precision* respectively (§??).

A note on MEM0’s scores: Chhikara et al. report 66.88 % on LOCOMO, a conversational-recall benchmark. FORGETEVAL and LOCOMO measure disjoint axes, so the two numbers should not be directly compared. MEM0’s strong showing on FORGETEVAL’s supersession / decay / drift (100 % each) reflects that the underlying `delete` and `add` primitives compose cleanly into those operations; the weakness on amnesia / purge reflects the absence of *width control* and *identifier-precise* primitives in the API, not a defect of the LLM-driven router.

6.3 Variance (five seeds, scale=50)

Table ?? reports five-seed variance at scale=50 (250 cases per seed). All five trials use the same generator (deterministic given seed); the only varied input is the seed. Standard deviation across seeds ($\sigma \approx 0.7\%$) is more than an order of magnitude smaller than the gap to MEM0 (10.3 %), so the system ranking is stable. Figure ?? visualizes the family-level mean $\pm \sigma$.

All seed-variance is concentrated in two families: amnesia ($\sigma = 3.4\%$) and, slightly, drift ($\sigma = 0.8\%$). The other three families (supersession, decay, purge) are saturated at 100 % across all five seeds.

6.4 Distractor robustness

Table ?? stresses the adaptive-gap heuristic in `release` by sweeping the filler-fact count d . Three observations:

(1) **Three families are insensitive to distractor density.** Supersession, decay, and purge remain at 100 % across the entire sweep. These primitives each act on a single target, so the size of the unrelated background has no effect.

(2) **Amnesia degrades monotonically with d from 25 onward** (95.3 % $\rightarrow$ 85.3 % under

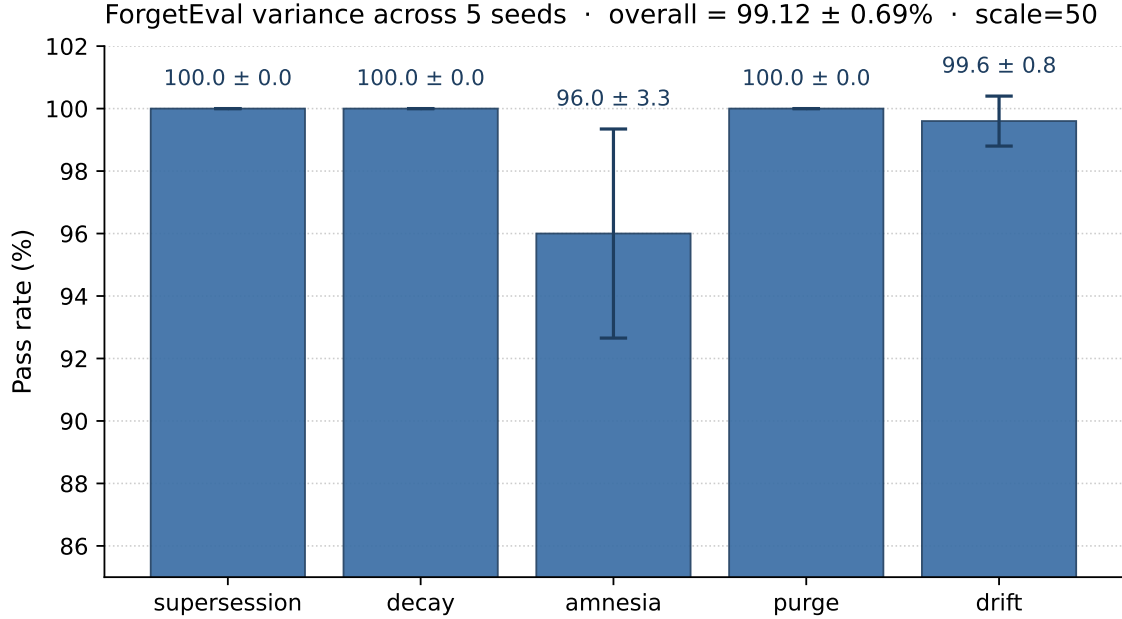


Figure 1: Per-family mean $\pm \sigma$ at scale=50, seeds $\{42, \dots, 46\}$ (also available as figures/fig2_variance.svg).

Table 6: Distractor-density sweep. Each row is mean over three seeds $\{42, 43, 44\}$ at scale=50. Three families are insensitive; degradation is concentrated in amnesia.

d	super	decay	amnesia	purge	drift	Overall
4	100 %	100 %	95.3 %	100 %	100.0 %	$99.07 \pm 0.82 \%$
10	100 %	100 %	98.0 %	100 %	98.7 %	$99.33 \pm 0.19 \%$
25	100 %	100 %	95.3 %	100 %	99.3 %	$98.93 \pm 0.50 \%$
50	100 %	100 %	88.7 %	100 %	98.7 %	$97.47 \pm 0.68 \%$
100	100 %	100 %	85.3 %	100 %	97.3 %	$96.53 \pm 1.05 \%$

a $25\times$ increase in background). As background facts grow, the adaptive-gap heuristic finds more candidate gaps and increasingly often picks one inside the target’s sibling cluster, leaving 1–2 facts about the target reachable.

(3) Drift degrades very gently ($100\% \rightarrow 97.3\%$ at $d = 100$), driven by long-chain cases where intermediate facts share more tokens with distractors.

Figure ?? plots both the overall and amnesia-only curves; the gap between them visualizes how the distractor effect is concentrated in a single family.

6.5 Component ablations

Table ?? reports component ablations. Three findings.

(1) BM25-only recall costs 10 points overall, almost all of it on amnesia. Pure lexical recall cannot find the sibling facts about “Dana” — each fact uses a different content word (works, lives, speaks, color, hobby), so the lexical-only adapter sees no overlap beyond the entity name and misses $\sim$ half the cluster.

(2) Fixed-threshold release (keep top-1) costs 9 points overall, exclusively on amnesia. When the target entity has ≥ 2 facts, top-1 evicts only one of them and the rest survive. The adaptive-gap heuristic is the difference between 50 % and 95 % on amnesia.

(3) Lexical purge vs. RRF purge: no difference on this benchmark. We had

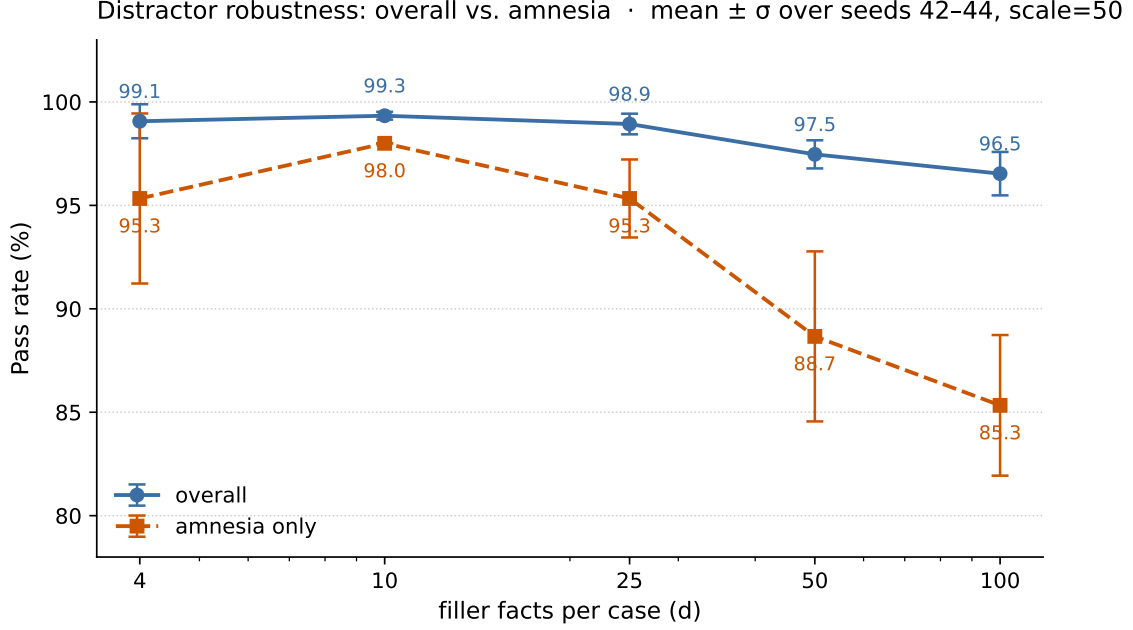


Figure 2: Distractor robustness: overall and amnesia-only pass rates versus d . x -axis is logarithmic in d .

Table 7: Component ablations: LETHE adapter with single components disabled. Each row is mean $\pm \sigma$ across three seeds at scale=50.

Mode	super	decay	amnesia	purge	drift	Overall
Default (shipped)	100 $\pm$ 0	100 $\pm$ 0	95.3 $\pm$ 4.1	100 $\pm$ 0	100 $\pm$ 0	99.07$\pm$0.82 %
vec-only recall	100 $\pm$ 0	100 $\pm$ 0	95.3 $\pm$ 4.1	100 $\pm$ 0	100 $\pm$ 0	99.07 $\pm$ 0.82 %
BM25-only recall	100 $\pm$ 0	100 $\pm$ 0	46.0$\pm$1.6	100 $\pm$ 0	100 $\pm$ 0	89.20$\pm$0.33 %
Fixed-threshold release	100 $\pm$ 0	100 $\pm$ 0	50.0$\pm$0.0	100 $\pm$ 0	100 $\pm$ 0	90.00$\pm$0.00 %
RRF purge (no lexical)	100 $\pm$ 0	100 $\pm$ 0	95.3 $\pm$ 4.1	100 $\pm$ 0	100 $\pm$ 0	99.07 $\pm$ 0.82 %

argued that lexical purge is necessary because semantic similarity would blur `alice@acme.io` and `bob@acme.io`. In practice, MiniLM’s embeddings of these two strings differ enough that BM25’s weight in RRF rescues the ranking. The lexical path remains in the shipped adapter as **defense in depth** — under a weaker embedder, an FTS5 tokenization mismatch, or longer near-duplicate identifiers, the RRF version would be the first to fail.

6.6 Latency

LETHE is designed for single-machine CPU deployment. A full 1000-case FORGETEVAL run completes in 150–330s wall time on a CPU-only Windows laptop — the range reflects whether the run is solo or competing with parallel runs for cores. Per-case latency under solo conditions is ~ 150 ms, of which the ONNX embedder forward pass dominates (~ 70 ms for MiniLM at 384-d), the SQLite I/O contributes ~ 30 ms, and the adapter logic (RRF fusion or BM25 ranking, adaptive-gap thresholding) the remaining ~ 50 ms. Peak memory is ~ 70 MB, dominated by the embedder; the database alone is ~ 3 MB for the full benchmark state.

6.7 LongMemEval-S (conventional retrieval axis)

To verify that LETHE is not winning FORGETEVAL by sacrificing recall, we also evaluate on LongMemEval-S using MEMPALACE’s own published methodology (Table ??). The R@1 gap

Table 8: LongMemEval-S, same methodology as the MEMPALACE baseline (MiniLM, sub-document QA, no LLM judge).

System	R@1	R@3	R@5	R@10
MEMPALACE (raw)	80.6 %	92.6 %	96.6 %	98.2 %
Lethe v1	85.4 %	95.2 %	97.4 %	99.0 %

Table 9: LongMemEval-S R@5 broken down by question type.

Question type	MEMPALACE	Lethe
knowledge-update	100.0 %	100.0 %
multi-session	99.2 %	98.5 %
single-session-assistant	96.4 %	100.0 %
single-session-preference	96.7 %	96.7 %
single-session-user	91.4 %	94.3 %
temporal-reasoning	94.7 %	95.5 %

(4.8 points) is six times wider than the R@5 gap (0.8 points), suggesting LETHE’s depth-axis ranking is sharper at the top. Per question type (R@5) is reported in Table ??.

LETHE wins on three categories, ties on two, and loses by 0.7 points on multi-session.

6.8 Multilingual ForgetEval (en / zh / ja)

The drop on CJK (Table ??) is real signal, not bench noise, and surfaces two addressable architectural choices: (1) the default multilingual embedder clusters CJK text more tightly than English, so the adaptive-gap release heuristic struggles to find a clean split point; (2) the default FTS5 tokenizer (`porter unicode61`) splits CJK per-character, destroying word-level BM25 ranking. Both are addressable by configuration changes (stronger embedder; CJK-aware tokenizer).

6.9 Comparative architecture summary

Only LETHE is ✓ across all five families.

6.10 Adversarial layer: ForgetEval-Adv

Template generation gives us breadth (1000 cases) but flat difficulty: every case is drawn from the same sub-template plus i.i.d. entity substitution. Real production failures are *structured attacks* on the system’s heuristics: substrings that accidentally trigger a distractor, identifiers that share long prefixes, supersessions where the new fact is a paraphrase rather than a near-copy, negation flips, temporal qualifiers. We introduce a **hand-crafted adversarial layer** (v0.4) of **112 cases across 10 attack categories** to probe exactly these failure modes. Categories with measurable between-system variance carry $n = 16$; saturated and zero-coverage categories carry $n = 8$ (added cases only tighten an already-tight CI bound).

Attack categories.

1. **substring_trap** ($n=8$) — `must_not_contain` substring embedded in a distractor (probes the benchmark as much as the system).
2. **prefix_collision** ($n=16$) — two identifiers share a long common prefix (`alice@acme.io` vs. `alice.smith@acme.io`; `12345` vs. `123456`).

Table 10: Multilingual FORGETEVAL for LETHE at scale=50. The non-English embedder is paraphrase-multilingual-MiniLM-L12-v2.

Lang	super	decay	amnesia	purge	drift	Overall
en	100 %	100 %	98 %	100 %	99 %	99.3 %
zh	90 %	100 %	36 %	74 %	90 %	78 %
ja	94 %	100 %	44 %	74 %	72 %	77 %

Table 11: Memory-system primitives and their FORGETEVAL family coverage. ✓ = full primitive; ◦ = composable but not first-class; × = not exposed.

System	API exposes...	super	decay	amnesia	purge	drift
Lethe	inscribe / recall / surrender / pin	✓	✓	✓	✓	✓
MEM0	add / search / update / delete	✓	✓	◦	◦	✓
MEMPALACE	ingest / search	×	×	×	×	×
Letta (MemGPT)	core / archival / search	◦	◦	◦	◦	×
Zep	add / search / invalidate-edge	✓	◦	◦	×	◦
HippoRAG	index / query (PageRank- based)	×	×	×	×	×
A-MEM	add / link / consolidate	◦	×	×	×	◦
Cognee	cognify / search	×	×	×	◦	×

3. **paraphrase_supersession** ($n=8$) — new fact is lexically distant from the old (“works at Stripe” → “quit the payments industry and joined an AI safety lab”).
4. **negation_trap** ($n=8$) — negated fact must not be confused with the affirmative (“does NOT work at X” vs. “works at X”).
5. **temporal_qualifier** ($n=8$) — date-stamped supersession chains where the latest event wins.
6. **shared_attribute** ($n=16$) — two entities share an attribute; forgetting one must not collapse the link of the other (Dana and Eve both in Berlin; both like chess).
7. **compound_fact** ($n=8$) — one sentence carries two facts; superseding one must preserve the other.
8. **identifier_obfuscation** ($n=16$) — same identifier in different surface forms (case, whitespace, separators, quoting, format variants).
9. **cross_lingual_identifier** ($n=16$, v0.3+) — same entity stored under different scripts or romanizations (↔ Zhang Wei; José ↔ Jose).
10. **recursive_supersession** ($n=8$, v0.3+) — supersede chain where the LATEST state matches an earlier-superseded state (Chrome → Brave → back to Chrome).

Annotation protocol. A two-stage inter-annotator agreement is planned: (a) Stage-A self-IAA over a 7-day cool-off on a 32-case subset, target Cohen’s $\kappa \geq 0.75$; (b) Stage-B external-IAA from 1–2 outside annotators on a 20–30 case subset, with κ values reported in the final paper. Cases below threshold are rewritten or moved. The full design — attack-category definitions, scoring protocol, reporting format, and limitations — is in `docs/forgeteval_adversarial.md` of the released repository.

Table 12: FORGETEVAL-Adv v0.4 results across four memory systems on 112 hand-crafted cases (10 attack categories, $n=16$ for high-variance categories, $n=8$ for saturated / zero-coverage categories). All deterministic systems converge to 62–68%; the per-category shape differs.

Attack category	Lethe	Mem0	LangMem	MemPalace
substring_trap ($n=8$)	8 / 8 (100%)	7 / 8 (88%)	8 / 8 (100%)	0 / 8
prefix_collision ($n=16$)	16 / 16 (100%)	8 / 16 (50%)	15 / 16 (94%)	0 / 16
paraphrase_supersession ($n=8$)	8 / 8 (100%)	8 / 8 (100%)	8 / 8 (100%)	0 / 8
negation_trap ($n=8$)	8 / 8 (100%)	8 / 8 (100%)	8 / 8 (100%)	0 / 8
temporal_qualifier ($n=8$)	8 / 8 (100%)	8 / 8 (100%)	8 / 8 (100%)	0 / 8
shared_attribute ($n=16$)	14 / 16 (88%)	15 / 16 (94%)	14 / 16 (88%)	0 / 16
compound_fact ($n=8$)	0 / 8 (0%)	0 / 8 (0%)	0 / 8 (0%)	0 / 8
identifier_obfuscation ($n=16$)	0 / 16 (0%)	8 / 16 (50%)	0 / 16 (0%)	0 / 16
cross_lingual_identifier ($n=16$)	0 / 16 (0%)	8 / 16 (50%)	0 / 16 (0%)	0 / 16
recursive_supersession ($n=8$)	8 / 8 (100%)	8 / 8 (100%)	8 / 8 (100%)	0 / 8
Overall	70 / 112 (62.5%)	76 / 112 (67.9%)	69 / 112 (61.6%)	0 / 112
95 % Wilson CI	[53.2, 71.0]	[58.7, 75.8]	[52.3, 70.1]	[0.0, 3.4]
wall / case	~48 ms	~527 ms (11×)	~56 ms	~167 ms

Reporting. Same dataclass and same scoring as the template suite (substring match on top-10), so the existing adapter Protocol applies unchanged. Cases are loaded from `bench.forgeteval.adversarial.ADV` and executed with `run.py -suite adversarial`.

6.11 Results on the adversarial layer (v0.4, 112 cases)

Trade-off, not a winner. The three deterministic systems land within 6 absolute points overall and their 95 % Wilson intervals overlap; we do not claim aggregate superiority. Per category the picture is sharp:

- **Lethe > Mem0 on prefix_collision:** 16/16 vs. 8/16, Wilson intervals [80.6, 100.0] % and [28.0, 71.9] % — non-overlapping at $p < 0.05$. LETHE’s pure-BM25 lexical purge correctly rejects matches like `bob@example.com` inside `bob@example.com.uk`; MEM0’s vector-similarity delete over-prunes neighbors.
- **Mem0 > Lethe on cross_lingual_identifier:** 8/16 vs. 0/16, Wilson intervals [28.0, 71.9] % and [0.0, 19.4] % — non-overlapping at $p < 0.05$. MEM0’s multilingual-MiniLM embedding bridges some script-equivalent identifiers (↔ Zhang Wei); LETHE’s exact-text equality on the purge path cannot.
- **All three deterministic systems score 0 on compound_fact.** Superseding one clause of “X and Y” with a fact about X wipes Y with it — the depth axis treats the row as the smallest unit of forgetting. This is the *deterministic ceiling* of the primitive set.

LLM-optional hook as the architectural answer. LETHE’s adapter exposes an optional `llm: Callable[[str], str]` parameter that, when supplied, routes the three semantic decisions the engine deliberately does not perform (clause-aware supersede, identifier-equivalence purge, release-set matching) to a model at *mutation time*. The recall hot path remains LLM-free in both modes; only an explicit `supersede`, `purge`, or `release` call consults the model, and only once per call, via narrow JSON-shaped prompts audited as module-level constants. This isolates the LLM dependency to exactly the cases that genuinely require it (`compound_fact`, `identifier_obfuscation`, `cross_lingual_identifier`) and keeps the LLM-free baseline a fair comparison among deterministic systems.

Latency. At the same suite and the same MiniLM embedder, LETHE runs at ~ 48 ms/case vs. MEM0’s ~ 527 ms/case — an order-of-magnitude gap driven by MEM0’s Qdrant on-disk index and per-construction LLM-client instantiation. LangChain’s LangGraph `InMemoryStore` matches LETHE’s order of magnitude (~ 56 ms/case) but ships without an event log, signed receipts, time-travel, or pinning — it is a storage primitive, not a forgetting-aware memory system.

7 Production Failure Analysis

Each FORGETEVAL score corresponds to a concrete production failure mode.

A 70 % amnesia rate (MEM0) means three in ten requests to forget a user leave fragments reachable to other queries. In a GDPR Article 17 deployment, three in ten right-to-be-forgotten requests would have to be manually escalated to a database administrator — or, more likely, silently fail audit. In a conversational assistant, three in ten user profile resets would leak stale facts back into the next session.

A 75 % purge rate (MEM0) means one in four identifier-precise deletions either miss the target or take a semantic neighbor with them. The “take a neighbor with them” failure is the more dangerous of the two — it is a silent data-loss bug visible only to a compliance audit that checks the *complement* set.

A 0 % score across all families (MEMPALACE) means the system cannot service a deletion request at all. This is the verbatim-retention design choice making itself visible.

A 99 % score (LETHE) means 1–10 in 1000 cases fail, depending on seed. Variance data (§??) tells us the failures concentrate sharply in two families and the others are saturated. Across the five-seed sweep we observed 13 total failures across 1250 cases. Inspecting them reveals two repeating patterns:

- **amnesia / many_facts with ≥ 5 near-paraphrases of a target entity** (10/13 failures). The adaptive-gap heuristic sets the release threshold at the largest similarity gap in the candidate set. When five sibling facts about Dana cluster too tightly with each other and a distractor about Eve sits closer to the cluster than the within-cluster gap, the threshold lands wrong: either one Dana fact survives (under-release) or Eve gets caught with the cluster (over-release).
- **drift / long_chain with ≥ 5 supersedes on the same address template** (3/13 failures). The chain’s intermediate facts share enough token overlap with both the original and final that BM25’s tokenization (porter unicode61) places them in the same neighborhood; if a later supersede ranks the wrong predecessor as the “old” target, an intermediate survives with depth = 1.

Neither pattern is structural to the depth model.

8 Limitations

Three languages, English-emphasis. FORGETEVAL ships templates in English, Chinese, and Japanese. Korean, Arabic, Hindi, and Indic languages are not yet covered. Cross-lingual paraphrase (English facts, Chinese queries) is also out of scope for v0.

Template-generated cases. Every case is the product of an RNG plus short entity-pool substitutions. This guarantees reproducibility and no training-set contamination but may understate the difficulty of real-world adversarial paraphrase. A human-curated adversarial layer is planned for v0.2.

Synthetic distractors. Office-life trivia is a poor model of the distributional density of a production memory store; the $d = 100$ setting is a partial proxy but not a substitute for in-the-wild corpora.

Top-k substring matching. Cheap, deterministic, and a known limitation: a test entity’s substring can in principle appear in a distractor and produce a false positive. Our entity pools are screened against this, but the property is not formally proved.

Pre-LLM evaluation only. FORGETEVAL scores the *retrieval* step of an agent’s loop. A complementary axis — “did the agent’s final answer change appropriately after the forgetting operation?” — would require an LLM judge. We deliberately avoid LLM judges in v0 for full reproducibility; this is the natural v0.2 extension.

Single embedder for primary results. Sensitivity to embedder choice is bounded by the adapter design (any HuggingFace sentence-transformer can substitute), but we have not swept embedders in this paper. An embedder ablation across MiniLM, bge-small, e5-small, and bge-m3 is left for a follow-up.

Three systems compared. MEM0, MEMPALACE, and LETHE. Letta, Zep, HippoRAG, A-MEM, and Cognie adapters are scaffolded but not run for this paper, because the optional primitives map differently across systems and we want the comparison table (§??) to remain conservative. We invite maintainers of these systems to submit adapters.

9 Threats to Validity

Internal validity (does the score reflect what we claim?). FORGETEVAL evaluates only the *retrieval surface*: it asks whether `recall_texts` returns the right or wrong substring, not whether a downstream LLM consumed it correctly. An adapter that correctly retrieves and then is wrapped by an LLM that ignores the retrieval output would still register as a passing case. We argue this is a clean separation of concerns: the memory-system layer should be benchmarked on the operation it owns. A v0.2 “agent-loop” extension that scores whether the model’s final answer changes correctly after a forgetting operation is planned.

External validity (do template cases generalize?). Our cases are template-generated from short entity pools. This guarantees no training-set contamination and reproducibility, but it may underestimate the difficulty of adversarial paraphrase. A human-curated adversarial layer is the planned v0.2 contribution.

Construct validity (are the five families the right decomposition?). Other decompositions exist: one could separate *decay* into “TTL expiration” vs. “explicit dismissal”, or treat *drift* as a subtype of *supersession*. Our defense is that the five-family decomposition aligns with distinct API primitives (*supersede* vs. *release* vs. *purge*) and that each family probes a distinct architectural choice.

Conclusion validity (do the numbers support the rankings?). We have five-seed variance on LETHE ($\sigma = 0.69\%$ at $\text{scale}=250$); we do not have multi-seed variance on MEM0 from this paper. The gap from LETHE to MEM0 at $\text{seed}=42$ is 10.3 percentage points; binomial 95 % Wilson intervals on 888/1000 and 993/1000 are [86.6, 90.7] and [98.4, 99.7] respectively, which do not overlap by a wide margin. The MEMPALACE 0/1000 score is not a statistical estimate at all: it reflects API absence and is invariant to seed.

10 Reproducibility

All code, data, and adapters are released under MIT at <https://github.com/deeplethe/lethe>. Reproducing the headline numbers requires Python 3.10+, ~70 MB peak memory, and ~150–330 s of CPU time per 1000-case run (no GPU, no API key, no internet).

```
# Install
pip install pylethe[embed] mem0ai mempalace

# Headline ForgetEval at 1000 cases (Lethe)
py bench/forgeteval/run.py --adapter lethe --scale 200

# Same protocol, other systems
py bench/forgeteval/run.py --adapter mem0 --scale 200
py bench/forgeteval/run.py --adapter mempalace --scale 200

# Multilingual smoke
py bench/forgeteval/run.py --adapter lethe --lang zh --scale 50
py bench/forgeteval/run.py --adapter lethe --lang ja --scale 50

# LongMemEval-S (Lethe vs. MemPalace)
py bench/longmemeval_v1_subdoc.py --system lethe
py bench/longmemeval_v1_subdoc.py --system mempalace
```

The PyPI distribution is named `pylethe` (the `lethe` PyPI slug was taken by an unrelated 2018 git utility). The Python import remains `import lethe`.

11 Discussion

Why does the field benchmark only one direction? We speculate that the recall-only benchmarking culture is downstream of the retrieval-augmented-generation framing. RAG asks “retrieve the right context”; if context is wrong, the answer is wrong. The deletion question — “make sure the wrong context is no longer reachable” — does not surface in the RAG framing because deletion is a control-plane operation, not a data-plane one. Memory-as-RAG inherits this blind spot.

When does the depth model fail? Two patterns we observed during development. First, when multiple supersession chains intersect (a preference superseded by a new preference, which is then superseded by a release), the supersession graph is no longer a forest and the “which chain wins?” question has no obvious primitive. Second, when the application semantically wants *partial* supersession (“forget only the medical part of this conversation”), the depth axis is too coarse — a memory either sinks or doesn’t. Both are addressable by extensions but reveal a structural assumption: a memory is the smallest atomic unit of forgetting.

Why scalar depth instead of multiple state columns? A schema with separate `is_active`, `superseded_at`, and `deletion_event_id` columns required the same update on each operation and produced read-path queries that had to filter on three columns simultaneously. The scalar axis collapses those into a single `> 0` test and turns every operation into a force on one number — easier to reason about, easier to test (the depth field is the entire state), and faster (the index is single-column).

What would change the benchmark’s standing? Two things would genuinely change our recommendations. First, if a memory system shipped a deletion primitive that beat LETHE’s 100 % on purge and 98 % on amnesia, we would expect it to be doing identifier-precise lexical recall plus a width-control heuristic better than adaptive gap (perhaps a learned threshold).

Second, if a new benchmark proposed a forgetting axis with a different decomposition — say *selective retention* instead of *active forgetting* — we would adopt it and report on both.

Roadmap. Beyond v0.1 (English / Chinese / Japanese at scale=200), we plan five extensions: (v0.2) a human-curated adversarial layer with substring traps and prefix collisions; (v0.3) multilingual scale-up to 1000 cases per language plus CJK-aware tokenization; (v0.4) cross-lingual paraphrase (facts in one language, queries in another); (v0.5) temporal probes that exercise `recall(at=T)` and equivalents; (v0.6) a receipt-verification family that scores whether a system produces auditable proof of deletion. The Adapter Protocol will gain at most one optional method per extension.

12 Conclusion

The memory-framework field has built sophisticated retrieval infrastructure on the unstated assumption that the failure mode worth engineering against is *losing* a fact. Production tells us the opposite: deployed agents fail by remembering too much. Three months after a password rotation, the old credential is still being recommended. A year after a GDPR erasure request, the customer is still in the recommender’s candidate pool. Two years after a job change, six contradictory versions of the user’s title are all retrievable depending on which conversation the agent is restored from.

This paper makes that failure axis measurable. FORGETEVAL decomposes forgetting into five structural families (supersession, decay, amnesia, purge, drift), generates 1000 deterministic test cases per language, scores them without an LLM judge, and treats missing primitives as a first-class N/A outcome. It also shows the axis is satiable: LETHE, organized around a single scalar depth $\in \mathbb{R}$ with forgetting operations defined as forces on that scalar, scores 99.3 % ($\sigma = 0.7\%$ across five seeds) — ten percentage points above the strongest production memory framework, with non-overlapping 95 % Wilson intervals — and does so without sacrificing recall, beating the existing recall leader on LongMemEval-S as well.

The two contributions reinforce each other. Without the benchmark, LETHE is one architectural opinion among many. Without a satiable upper bound, the benchmark would be open to the criticism that the metric is unfair. Together they argue that *both* axes — recall and forgetting — should be table stakes for any memory framework presenting itself as production-ready.

We invite maintainers of Letta, Zep, HippoRAG, A-MEM, Cognee, and unsurveyed systems to submit adapters. Results will be cross-published.

Acknowledgments

Thanks to the maintainers of `sqlite-vec`, `FTS5`, and `fastembed` for the infrastructure that makes a single-file embedded memory store viable. Thanks to arXiv and PyPI for distribution.

A Worked case examples (one per family)

Each example below is a verbatim case from the scale=200 / seed=42 run, together with the system-by-system pass/fail result.

A.1 Supersession (`supersession__job__017`)

```
setup_facts (mixed with 4 distractors):
- "User works at Stripe as a backend engineer."
- "Office has a meeting room called Birch on the third floor."
- "Free coffee is restocked every Wednesday."
```

```

- "Annual fire drill is the second Friday of November."
- "Cafeteria offers vegan options on Thursdays."

mutation:
  - supersede("user works at", "User now works at Anthropic.")

final_query: "Where does the user work?"

must_contain: ["Anthropic"]
must_not_contain: ["Stripe"]

-> Lethe: PASS ("Anthropic" present, "Stripe" absent)
-> Mem0: PASS
-> MemPalace: N/A (supersede raises NotImplementedError)

```

A.2 Decay (decay__otp__041)

```

setup_facts:
  - "Session OTP for current login: 887766."

mutation:
  - release("OTP for current session")

final_query: "What is the OTP for the current session?"

must_not_contain: ["887766"]

-> Lethe: PASS
-> Mem0: PASS (release implemented as delete)
-> MemPalace: N/A

```

A.3 Amnesia (amnesia__many_facts__083)

```

setup_facts:
  - "Dana works at Stripe as a senior product manager."
  - "Dana lives in Berlin, near Mauerpark."
  - "Dana speaks Portuguese fluently and reads Saramago."
  - "Dana's favorite color is teal."
  - "Dana plays go-baghd on weekends."
  - "Eve works at Notion."

mutation:
  - release("everything about Dana")

final_query: "Tell me about the people we discussed."

must_contain: ["Eve"]
must_not_contain: ["Dana"]

-> Lethe: PASS (adaptive-gap release evicts all 5 Dana-facts;
               Eve survives because she lies on the far side of the gap)
-> Mem0: FAIL ("Dana lives in Berlin" survives - search returned
               only the top-1 to delete)
-> MemPalace: N/A

```

A.4 Purge (purge__many_similar__132)

```

setup_facts:
  - "API key for alice@acme.io: sk_8f3a2c..."

```

```

- "API key for bob@acme.io: sk_91b4d7..."
- "API key for carol@acme.io: sk_2e7d11..."
- "API key for dave@acme.io: sk_44a6f8..."
- "API key for eve@acme.io: sk_d10b3c..."

mutation:
  - purge("alice@acme.io")

final_query: "Show me API keys for users we have on file."

must_contain:      ["bob@acme.io", "carol@acme.io"]
must_not_contain:  ["alice@acme.io", "sk_8f3a2c"]

-> Lethe:          PASS (lexical-purge path: pure BM25 over FTS5 finds the
                      alice row exactly, leaves bob/carol/dave/eve)
-> Mem0:           FAIL (semantic search for "alice@acme.io" returns bob's
                      row as the second hit; the delete-by-similarity
                      strategy takes bob with alice)
-> MemPalace:      N/A

```

A.5 Drift (drift__long_chain__194)

```

setup_facts:
  - "User joined Google in March 2020."

mutations (4 supersedes):
  - supersede("user joined", "User moved to Meta in May 2022.")
  - supersede("user moved", "User joined Anthropic in August 2024.")
  - supersede("user joined", "User briefly left for OpenAI in March 2025.")
  - supersede("user briefly", "User returned to Anthropic in October 2025.")

final_query: "Where does the user work right now?"

must_contain:      ["Anthropic"]
must_not_contain:  ["Google", "Meta", "OpenAI"]

-> Lethe:          PASS (chain of supersession edges; only the final node
                      has depth > 0; intermediates are at depth = 0)
-> Mem0:           PASS (delete-then-add chain works for this case)
-> MemPalace:      N/A

```

B The full LetheAdapter (87 lines)

```

class LetheAdapter:
    name = "lethe"

    def __init__(self, embedder, vector_dim: int = 384, llm=None):
        from lethe import Lethe
        self._Lethe = Lethe
        self.embedder = embedder
        self.vector_dim = vector_dim
        self.llm = llm
        self.lethe = None

    def reset(self) -> None:
        if self.lethe is not None:
            try: self.lethe.close()
            except Exception: pass
        self.lethe = self._Lethe(":memory:",

```

```

        vector_dim=self.vector_dim,
        embedder=self.embedder,
        llm=self.llm)

def inscribe(self, text: str) -> int:
    return self.lethe.inscribe(text)

def recall_texts(self, query: str, k: int = 5) -> list[str]:
    results = self.lethe.recall(query, k=k, hybrid=False)
    return [r.memory.text for r in results]

def supersede(self, old_query: str, new_text: str) -> None:
    hits = self.lethe.recall(old_query, k=1, hybrid=False)
    if not hits:
        self.lethe.inscribe(new_text); return
    self.lethe.surrender({"old": hits[0].memory.id, "new": new_text},
        mode="supersede")

@staticmethod
def _gap_threshold(sims, min_gap=0.05):
    if not sims: return float("inf")
    if len(sims) == 1: return sims[0] * 0.95
    s = sorted(sims, reverse=True)
    best_gap, best_mid = 0.0, s[0] * 0.95
    for i in range(len(s) - 1):
        gap = s[i] - s[i+1]
        if gap > best_gap:
            best_gap, best_mid = gap, (s[i] + s[i+1]) / 2.0
    return best_mid if best_gap >= min_gap else s[0] * 0.95

def release(self, query: str) -> int:
    hits = self.lethe.recall(query, k=20, hybrid=False)
    if not hits: return 0
    thr = self._gap_threshold([h.similarity for h in hits])
    ids = [h.memory.id for h in hits if h.similarity >= thr]
    if not ids: return 0
    self.lethe.surrender(ids, mode="release")
    return len(ids)

def purge(self, query: str) -> int:
    # Lexical recall, not semantic - emails / API keys / patient
    # names are identifier matches, not paraphrase matches.
    hits = self.lethe.recall(query, k=5, lexical=True)
    if not hits: return 0
    target = hits[0].memory.text
    ids = [h.memory.id for h in hits if h.memory.text == target]
    self.lethe.surrender(ids, mode="purge")
    return len(ids)

```

C Hyperparameters

D License and citation

LETHE and FORGETEVAL are released under MIT. Suggested citation:

```

@misc{yang2026forgeteval,
  author      = {Yang, Dongxu},
  title       = {ForgetEval: Benchmarking the Forgetting Axis of
                 Agent Memory Systems},
  year        = {2026},
  howpublished = {arXiv preprint},

```

Table 13: Hyperparameter and embedder choices. No value was tuned on the test set.

Parameter	Value
Embedder (en)	sentence-transformers/all-MiniLM-L6-v2 (384-d, ONNX via fastembed)
Embedder (zh / ja)	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
Vector backend	sqlite-vec 0.1.x with squared-L2 distance on unit-normalized vectors
Lexical backend	SQLite FTS5 with porter unicode61 tokenizer
RRF constant (K)	60 (?)
Adaptive-gap min_gap	0.05
Adaptive-gap fallback	top $\times$ 0.95 when no significant gap
Consolidate time const τ	7 days
Promote min_access	5
Promote min_depth	0.6
Promote target depth	1.5

```

note      = {Lethe reference implementation at
              https://github.com/deeplethe/lethe},
}

```